JÚLIO ZADI
LUAN FLÔRES MARTINS
LENINI BELLODI JÚNIOR
JULIANO ARAUJO
JOÃO VICTOR MULLER MIRANDA

SMARTCONDO
Sistema de Gerenciamento de Condomínios

Implementação do banco de dados — 2ª parte:
consultas, visões, funções, procedimentos, gatilhos e segurança

Trabalho apresentado à disciplina de Laboratório de Banco de Dados, como parte da avaliação EC 2.

Professor: Odirley Franco

Campo Grande – MS
2026

1 INTRODUÇÃO

O SmartCondo é um sistema web de gestão de condomínios, com telas para o administrador da plataforma, o síndico, a portaria e os moradores. Os dados ficam num banco **PostgreSQL 16**, e todos os scripts deste trabalho foram escritos para rodar no **pgAdmin**, abrindo o arquivo e executando com F5.

Na 1ª parte do projeto, foram apresentadas a criação das tabelas (script 01) e a carga de um condomínio de demonstração (script 02). Esta 2ª parte implementa, sobre esse banco, o que faz dele mais do que um depósito de dados: consultas que respondem às perguntas do dia a dia do condomínio, visões, funções e procedimentos armazenados, gatilhos, um índice, o controle de acesso por papéis e o uso de transações.

Antes de construir a 2ª parte, o banco foi conferido de novo, para garantir que a base está correta (seção 2). Os resultados mostrados ao longo do texto foram obtidos executando os scripts num banco vazio, em 12/11/2026.

2 O BANCO DE DADOS

2.1 Números

Desde a 1ª parte, o banco cresceu junto com o sistema. Hoje, criado pelo script 01:

Item	Quantidade
Tabelas	40
Colunas	489
Chaves primárias (uma em cada tabela)	40
Chaves estrangeiras	115
Restrições CHECK	35
Restrições UNIQUE	7
Índices (inclusive os das chaves e os únicos parciais)	159
Tipos enumerados (ENUM)	28
Registros na carga de demonstração (script 02)	184

As tabelas se agrupam em seis áreas: **cadastro** (condomínios, unidades, usuários, permissões do porteiro), **financeiro** (cobranças, pagamentos, acordos de parcelamento, despesas, preferências de cobrança), **portaria** (visitantes, autorizações de visita, encomendas, veículos, chaves, livro da portaria), **áreas comuns** (espaços, reservas, ocupação), **gestão** (assembleias, pauta e votos, comunicados, documentos, ocorrências, ordens de serviço, manutenção preventiva, infrações, mudanças, reformas) e **auditoria** (registro de alterações).

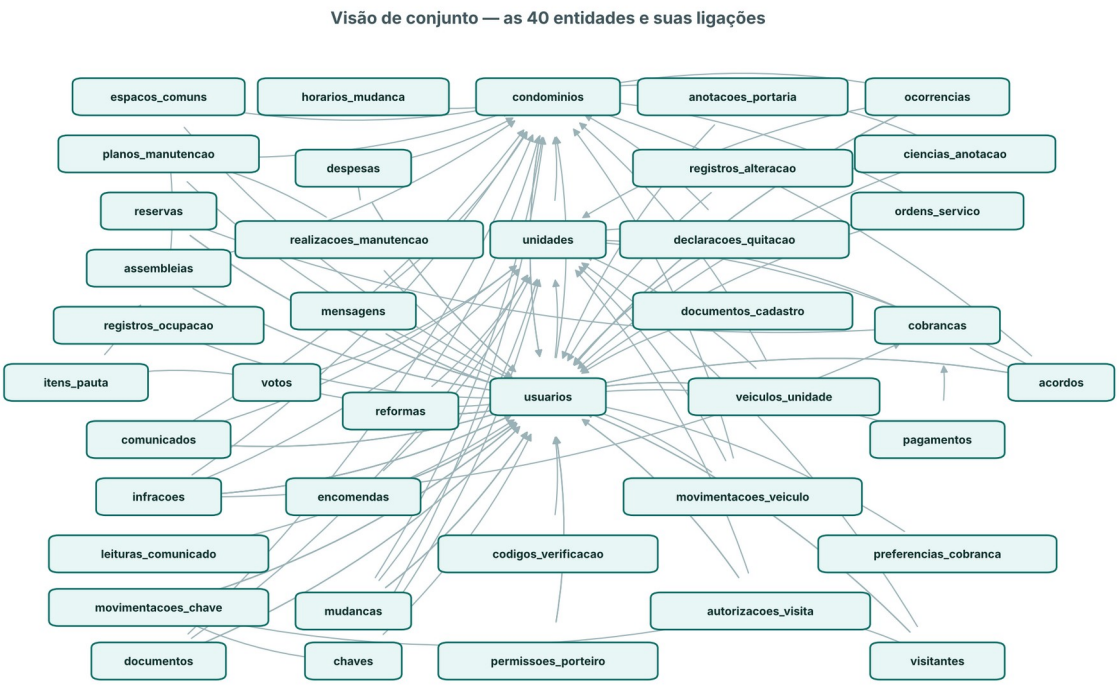


Figura 1 – Diagrama entidade-relacionamento: visão geral das tabelas e das ligações

2.2 Conferência do banco

A conferência foi feita em um banco vazio, executando os scripts como o pgAdmin executa: o arquivo inteiro de uma vez.

O que foi conferido	Resultado
Scripts 01 e 02 em banco vazio	Executam sem erro
Executar os scripts de novo sobre o banco já criado	Executam sem erro e deixam o mesmo banco: são reexecutáveis
Estrutura criada pelo script 01 × estrutura que o sistema usa (migrações do back-end)	Idênticas: tabelas, colunas, tipos, chaves, índices e restrições, comparados por programa
Toda tabela tem chave primária	40 de 40
Registros órfãos (chave estrangeira apontando para o que não existe)	Nenhum: as chaves estrangeiras são conferidas pelo próprio banco na carga
Dados inválidos são recusados	CHECK, chave estrangeira e índice único barram (seção 8)
Senhas	Guardadas só em hash bcrypt, nunca em texto

Desde a 1ª parte, entraram no banco, entre outras, as tabelas de reformas, acordos de parcelamento, chaves da portaria, manutenção preventiva e declarações de quitação. Nesta etapa, a tabela **unidades** ganhou a coluna **fracao_ideal** (a fração ideal da convenção, em %, com CHECK de 0 a 100), usada no rateio das despesas, e o tipo **origem_cobranca** ganhou o valor **EXTRA**, para as cotas extras aprovadas em assembleia.

3 SCRIPTS E ORDEM DE EXECUÇÃO

Ordem	Arquivo	O que faz	Executar como
1	01_criar_tabelas.sql	Cria os tipos, as tabelas, as chaves e os índices	dono do banco
2	02_carga_dados.sql	Carrega o condomínio de demonstração	dono do banco
3	03_consultas.sql	As 14 consultas da seção 4 (só leitura)	qualquer um
4	04_programacao.sql	Visões, funções, procedimentos, gatilhos, índice e a tabela de log	dono do banco
5	05_seguranca.sql	Papéis de acesso e permissões	postgres
6	06_demonstracao.sql	Usa tudo dentro de uma transação desfeita no fim	postgres

Todos os scripts são reexecutáveis: cada um apaga o que criou antes de recriar. O 05 e o 06 pedem o usuário postgres porque criam papéis e trocam de papel (SET ROLE).

4 CONSULTAS

As consultas do script 03 respondem a perguntas reais do síndico, da portaria e da assembleia, e cada uma exercita um recurso diferente da linguagem SQL. As que dependem da data de hoje (atraso, dias restantes) mudam conforme o dia da execução.

4.1 Moradores ativos e seus apartamentos

Pergunta: Quem mora em cada apartamento, e se é proprietário ou inquilino?

Recursos de SQL: junção interna (JOIN), filtro (WHERE), CASE e ordenação.

```
SELECT u.numero AS apartamento,
       u.andar,
       m.nome AS morador,
       m.tipo_ocupacao,
       CASE WHEN m.reside THEN 'sim' ELSE 'não' END AS mora_no_apartamento
  FROM usuarios m
  JOIN unidades u ON u.id = m.unidade_id
 WHERE m.papel = 'MORADOR'
   AND m.status = 'ATIVO'
 ORDER BY u.numero, m.nome;
```

Resultado (6 linhas)

apartamento	andar	morador	tipo_ocupacao	mora_no_apartamento
101	1	Roberto Nascimento	PROPRIETARIO	sim
102	1	Bruno Cardoso	INQUILINO	sim
204	2	João Silva	PROPRIETARIO	sim
301	3	Ana Beatriz Rocha	PROPRIETARIO	sim
302	3	João Silva	PROPRIETARIO	não
410	4	Marina Duarte	PROPRIETARIO	sim

O João Silva aparece duas vezes: é proprietário do 204, onde mora, e do 302, que tem mas onde não mora.

4.2 Apartamentos sem morador residindo

Pergunta: Quais apartamentos estão sem ninguém morando, segundo o cadastro?

Recursos de SQL: subconsulta correlacionada com NOT EXISTS.

```
SELECT u.numero AS apartamento,
       u.andar,
       u.vagas_garagem
  FROM unidades u
 WHERE NOT EXISTS (
         SELECT 1
           FROM usuarios m
          WHERE m.unidade_id = u.id
            AND m.papel = 'MORADOR'
            AND m.status = 'ATIVO'
            AND m.reside
       )
 ORDER BY u.numero;
```

Resultado (2 linhas)

apartamento	andar	vagas_garagem
302	3	1
502	5	2

O 302 tem dono, mas o dono mora no 204; o 502 tem um inquilino cujo cadastro ainda aguarda aprovação do síndico.

4.3 Situação financeira por apartamento

Pergunta: Quanto cada apartamento já foi cobrado, quanto pagou e quanto deve?

Recursos de SQL: junção externa (LEFT JOIN), tabela derivada na cláusula FROM, SUM, COUNT, COALESCE, FILTER e GROUP BY.

```
SELECT u.numero AS apartamento,
       COUNT(c.id) AS cobrancas,
       COALESCE(SUM(c.valor), 0) AS total_cobrado,
       COALESCE(SUM(p.pago), 0) AS total_pago,
       COALESCE(SUM(c.valor - COALESCE(p.pago, 0))
                FILTER (WHERE c.status IN ('ABERTA', 'VENCIDA')), 0) AS em_aberto
  FROM unidades u
  LEFT JOIN cobrancas c
         ON c.unidade_id = u.id
        AND c.status NOT IN ('CANCELADA', 'NEGOCIADA')
  LEFT JOIN (SELECT cobranca_id, SUM(valor - encargos) AS pago
               FROM pagamentos
              GROUP BY cobranca_id) p
         ON p.cobranca_id = c.id
 GROUP BY u.id, u.numero
 ORDER BY u.numero;
```

Resultado (7 linhas)

apartamento	cobrancas	total_cobrado	total_pago	em_aberto
101	3	960.00	640.00	320.00
102	3	960.00	640.00	320.00
204	4	1110.00	640.00	470.00
301	3	960.00	640.00	320.00

apartamento	cobrancas	total_cobrado	total_pago	em_aberto
302	3	960.00	320.00	640.00
410	6	1640.00	1093.34	546.66
502	0	0	0	0

O LEFT JOIN mantém o 502, que não tem cobrança; sem ele, o apartamento sumiria do resultado. O pago soma o valor dos pagamentos menos a multa e os juros, que não abatem a dívida.

4.4 Apartamentos em atraso

Pergunta: Quem está devendo hoje, quanto e há quantos dias?

Recursos de SQL: aritmética de datas (CURRENT_DATE - vencimento), GROUP BY e HAVING.

```
SELECT u.numero AS apartamento,
       COUNT(*) AS cobrancas_em_atraso,
       SUM(c.valor - COALESCE(p.pago, 0)) AS valor_devido,
       MAX(CURRENT_DATE - c.vencimento) AS dias_de_atraso
  FROM cobrancas c
  JOIN unidades u ON u.id = c.unidade_id
  LEFT JOIN (SELECT cobranca_id, SUM(valor - encargos) AS pago
               FROM pagamentos
              GROUP BY cobranca_id) p
         ON p.cobranca_id = c.id
 WHERE c.status IN ('ABERTA', 'VENCIDA')
   AND c.vencimento < CURRENT_DATE
 GROUP BY u.numero
HAVING SUM(c.valor - COALESCE(p.pago, 0)) > 0
 ORDER BY valor_devido DESC, u.numero;
```

Resultado (6 linhas)

apartamento	cobrancas_em_atraso	valor_devido	dias_de_atraso
302	2	640.00	94
410	2	546.66	63
204	2	470.00	63
101	1	320.00	63
102	1	320.00	63
301	1	320.00	63

Como os dados de demonstração foram carregados em setembro, as cobranças de setembro já venceram hoje: o resultado muda conforme o dia em que a consulta é executada.

4.5 Inadimplência por competência

Pergunta: Em cada mês, quanto foi cobrado de taxa, quanto entrou e qual a inadimplência?

Recursos de SQL: agregação com porcentagem, ROUND e to_char.

```
SELECT to_char(c.competencia, 'MM/YYYY') AS competencia,
       SUM(c.valor) AS cobrado,
       COALESCE(SUM(p.pago), 0) AS recebido,
       ROUND(100 * (1 - COALESCE(SUM(p.pago), 0) / SUM(c.valor)), 1) AS inadimplencia_pct
  FROM cobrancas c
  LEFT JOIN (SELECT cobranca_id, SUM(valor - encargos) AS pago
               FROM pagamentos
              GROUP BY cobranca_id) p
         ON p.cobranca_id = c.id
 WHERE c.origem = 'TAXA'
   AND c.status NOT IN ('CANCELADA', 'NEGOCIADA')
 GROUP BY c.competencia
 ORDER BY c.competencia;
```

Resultado (3 linhas)

competencia	cobrado	recebido	inadimplencia_pct
07/2026	2010.00	2010.00	0.0
08/2026	2010.00	1690.00	15.9
09/2026	2010.00	0	100.0

4.6 Despesas por categoria

Pergunta: Para onde vai o dinheiro do condomínio?

Recursos de SQL: função de janela SUM(...) OVER () sobre um agrupamento: a participação de cada categoria no total.

```
SELECT categoria,
       COUNT(*) AS lancamentos,
       SUM(valor) AS total,
       ROUND(100 * SUM(valor) / SUM(SUM(valor)) OVER (), 1) AS participacao_pct
  FROM despesas
 WHERE inativo_em IS NULL
 GROUP BY categoria
 ORDER BY total DESC;
```

Resultado (5 linhas)

categoria	lancamentos	total	participacao_pct
LIMPEZA	3	2340.00	47.0
MANUTENCAO	1	980.00	19.7
ENERGIA	3	937.11	18.8
AGUA	3	596.70	12.0
ADMINISTRACAO	3	119.70	2.4

A limpeza leva quase metade das despesas do período.

4.7 Balancete mensal com saldo acumulado

Pergunta: Mês a mês, quanto entrou, quanto saiu e qual o saldo?

Recursos de SQL: expressão de tabela comum (WITH), FULL JOIN e soma acumulada com janela ordenada (OVER (ORDER BY ...)).

```
WITH receitas AS (
       SELECT date_trunc('month', pago_em AT TIME ZONE 'America/Campo_Grande')::date AS
mes,
              SUM(valor) AS valor
         FROM pagamentos
        GROUP BY 1
     ),
     gastos AS (
       SELECT date_trunc('month', data)::date AS mes,
              SUM(valor) AS valor
         FROM despesas
        WHERE inativo_em IS NULL
        GROUP BY 1
     )
SELECT to_char(COALESCE(r.mes, g.mes), 'MM/YYYY') AS mes,
       COALESCE(r.valor, 0) AS receitas,
       COALESCE(g.valor, 0) AS despesas,
       COALESCE(r.valor, 0) - COALESCE(g.valor, 0) AS saldo_do_mes,
       SUM(COALESCE(r.valor, 0) - COALESCE(g.valor, 0))
           OVER (ORDER BY COALESCE(r.mes, g.mes)) AS saldo_acumulado
  FROM receitas r
  FULL JOIN gastos g ON g.mes = r.mes
 ORDER BY COALESCE(r.mes, g.mes);
```

Resultado (3 linhas)

mes	receitas	despesas	saldo_do_mes	saldo_acumulado
07/2026	2146.67	1331.17	815.50	815.50
08/2026	1826.67	2311.17	-484.50	331.00
09/2026	0	1331.17	-1331.17	-1000.17

O FULL JOIN é necessário porque pode haver mês com despesa e sem receita (setembro) ou o contrário.

4.8 Uso dos espaços comuns

Pergunta: Quais espaços são mais reservados e quantas horas ficaram ocupados?

Recursos de SQL: LEFT JOIN (espaços sem reserva também aparecem), COUNT com FILTER e EXTRACT sobre intervalo de horas.

```
SELECT e.nome AS espaco,
       e.capacidade,
       e.taxa_uso,
       COUNT(r.id) AS reservas,
       COUNT(r.id) FILTER (WHERE r.status = 'APROVADA') AS aprovadas,
       COUNT(r.id) FILTER (WHERE r.status = 'PENDENTE') AS pendentes,
       COUNT(r.id) FILTER (WHERE r.status = 'CONCLUIDA') AS concluidas,
       ROUND(COALESCE(SUM(EXTRACT(EPOCH FROM r.hora_fim - r.hora_inicio) / 3600)
                      FILTER (WHERE r.status IN ('APROVADA', 'CONCLUIDA')), 0), 1) AS
horas_ocupadas
  FROM espacos_comuns e
  LEFT JOIN reservas r ON r.espaco_id = e.id
 WHERE e.reservavel
   AND e.inativo_em IS NULL
 GROUP BY e.id, e.nome, e.capacidade, e.taxa_uso
 ORDER BY reservas DESC, e.nome;
```

Resultado (5 linhas)

espaco	capacidade	taxa_uso	reservas	aprovadas	pendentes	concluidas	horas_ocupadas
Salão de Festas	80	150.00	2	1	0	1	13.0
Churrasqueira 2	20	60.00	1	0	1	0	0.0
Salão de Jogos	25		1	0	1	0	0.0
Churrasqueira 1	20	60.00	0	0	0	0	0.0
Coworking	10		0	0	0	0	0.0

4.9 Resultado da assembleia

Pergunta: Cada item da pauta foi aprovado, segundo a maioria que exige?

Recursos de SQL: CROSS JOIN LATERAL, contagens com FILTER e CASE com as três regras de maioria (simples, dois terços e unanimidade).

```
SELECT i.ordem,
       i.titulo,
       i.maioria,
       COUNT(v.id) FILTER (WHERE v.escolha = 'A_FAVOR') AS a_favor,
       COUNT(v.id) FILTER (WHERE v.escolha = 'CONTRA') AS contra,
       COUNT(v.id) FILTER (WHERE v.escolha = 'ABSTENCAO') AS abstencao,
       t.unidades,
       CASE i.maioria
         WHEN 'SIMPLES' THEN
           COUNT(v.id) FILTER (WHERE v.escolha = 'A_FAVOR')
             > COUNT(v.id) FILTER (WHERE v.escolha = 'CONTRA')
         WHEN 'DOIS_TERCOS' THEN
           3 * COUNT(v.id) FILTER (WHERE v.escolha = 'A_FAVOR') >= 2 * t.unidades
         ELSE
           COUNT(v.id) FILTER (WHERE v.escolha = 'A_FAVOR') = t.unidades
       END AS aprovado
  FROM assembleias a
  JOIN itens_pauta i ON i.assembleia_id = a.id
  LEFT JOIN votos v ON v.item_id = i.id
 CROSS JOIN LATERAL (
         SELECT COUNT(*) AS unidades
           FROM unidades u
          WHERE u.condominio_id = a.condominio_id
            AND NOT u.pendente
       ) t
 WHERE a.id = 1
 GROUP BY i.id, i.ordem, i.titulo, i.maioria, t.unidades
 ORDER BY i.ordem;
```

Resultado (2 linhas)

ordem	titulo	maioria	a_favor	contra	abstencao	unidades	aprovado
1	Aprovar as contas do semestre	SIMPLES	3	0	1	7	true
2	Pintura da fachada (R$ 42 mil, rateados em 6 parcelas)	DOIS_TERCOS	3	1	0	7	false

A pintura da fachada teve 3 votos a favor e 1 contra, mas exige dois terços dos 7 apartamentos (5 votos): foi reprovada. A aprovação das contas, por maioria simples, passou.

4.10 Participação na assembleia

Pergunta: Quais apartamentos votaram?

Recursos de SQL: subconsulta com IN.

```
SELECT u.numero AS apartamento,
       CASE WHEN u.id IN (SELECT v.unidade_id
                            FROM votos v
                            JOIN itens_pauta i ON i.id = v.item_id
                           WHERE i.assembleia_id = 1)
            THEN 'votou'
            ELSE 'não votou'
       END AS participacao
  FROM unidades u
 WHERE u.condominio_id = 1
 ORDER BY u.numero;
```

Resultado (7 linhas)

apartamento	participacao
101	não votou
102	votou
204	votou
301	votou
302	não votou
410	votou
502	não votou

4.11 Rateio da obra pela fração ideal

Pergunta: Se a pintura da fachada (R$ 42 mil, em 6 parcelas) for aprovada, quanto cabe a cada apartamento?

Recursos de SQL: função de janela SUM(fracao_ideal) OVER () para dividir pela soma das frações.

```
SELECT numero AS apartamento,
       fracao_ideal,
       ROUND(42000 * fracao_ideal / SUM(fracao_ideal) OVER (), 2) AS parte_na_obra,
       ROUND(42000 * fracao_ideal / SUM(fracao_ideal) OVER () / 6, 2) AS parcela_mensal
  FROM unidades
 WHERE condominio_id = 1
 ORDER BY numero;
```

Resultado (7 linhas)

apartamento	fracao_ideal	parte_na_obra	parcela_mensal
101	13.5000	5670.00	945.00
102	13.5000	5670.00	945.00
204	15.5000	6510.00	1085.00
301	13.5000	5670.00	945.00
302	13.5000	5670.00	945.00
410	15.5000	6510.00	1085.00
502	15.0000	6300.00	1050.00

As partes somam exatamente R$ 42.000,00, e as parcelas, R$ 7.000,00 por mês.

4.12 Pendências da portaria

Pergunta: O que a portaria tem em aberto agora: encomendas para entregar e visitantes dentro do condomínio?

Recursos de SQL: UNION ALL de duas consultas, subconsulta na cláusula FROM e fuso horário (AT TIME ZONE).

```
SELECT tipo, apartamento, descricao,
       to_char(desde AT TIME ZONE 'America/Campo_Grande', 'DD/MM/YYYY HH24:MI') AS desde,
       CURRENT_DATE - (desde AT TIME ZONE 'America/Campo_Grande')::date AS dias
  FROM (
         SELECT 'Encomenda para retirar' AS tipo, u.numero AS apartamento,
                e.remetente AS descricao, e.recebida_em AS desde
           FROM encomendas e
           JOIN unidades u ON u.id = e.unidade_id
          WHERE e.status = 'AGUARDANDO_RETIRADA'
         UNION ALL
         SELECT 'Visitante dentro', u.numero, v.nome, v.entrada_em
           FROM visitantes v
           JOIN unidades u ON u.id = v.unidade_id
          WHERE v.status = 'DENTRO'
       ) pendencias
 ORDER BY pendencias.desde;
```

Resultado (2 linhas)

tipo	apartamento	descricao	desde	dias
Encomenda para retirar	204	Correios	20/09/2026 18:45	53
Visitante dentro	301	Fernanda Souza	20/09/2026 19:45	53

4.13 Manutenções preventivas

Pergunta: Quais manutenções obrigatórias estão vencidas ou perto de vencer?

Recursos de SQL: CASE com três situações e diferença entre datas.

```
SELECT nome,
       periodicidade_meses,
       proximo_vencimento,
       proximo_vencimento - CURRENT_DATE AS dias_restantes,
       CASE
         WHEN proximo_vencimento < CURRENT_DATE THEN 'VENCIDA'
         WHEN proximo_vencimento - CURRENT_DATE <= antecedencia_aviso_dias THEN 'AVISAR'
         ELSE 'EM DIA'
       END AS situacao
  FROM planos_manutencao
 WHERE inativo_em IS NULL
 ORDER BY proximo_vencimento;
```

Resultado (5 linhas)

nome	periodicidade_meses	proximo_vencimento	dias_restantes	situacao
Recarga dos extintores	12	2026-09-10	-63	VENCIDA
Limpeza da caixa d'água	6	2026-10-10	-33	VENCIDA
Manutenção dos elevadores	1	2026-10-15	-28	VENCIDA
Inspeção do para-raios (SPDA)	12	2027-05-23	192	EM DIA
AVCB (vistoria do Corpo de Bombeiros)	36	2027-07-17	247	EM DIA

Extintores, caixa d’água e elevadores estão vencidos: o síndico precisa agir.

4.14 Últimas alterações (auditoria)

Pergunta: Quem fez o quê no sistema, por último?

Recursos de SQL: JOIN com a tabela de usuários, ordenação decrescente e LIMIT.

```
SELECT to_char(r.feito_em AT TIME ZONE 'America/Campo_Grande', 'DD/MM/YYYY HH24:MI') AS
quando,
       us.nome AS quem,
       r.acao,
       r.entidade,
       r.entidade_id,
       r.descricao
  FROM registros_alteracao r
  JOIN usuarios us ON us.id = r.autor_id
 ORDER BY r.feito_em DESC, r.id DESC
 LIMIT 10;
```

Resultado (10 linhas)

quando	quem	acao	entidade	entidade_id	descricao
12/11/2026 15:32	Roberto Nascimento	criou	espaco	8	
12/11/2026 15:32	Roberto Nascimento	criou	espaco	7	
12/11/2026 15:32	Roberto Nascimento	criou	espaco	6	
12/11/2026 15:32	Roberto Nascimento	criou	espaco	5	
12/11/2026 15:32	Roberto Nascimento	criou	espaco	4	
12/11/2026 15:32	Roberto Nascimento	criou	espaco	3	
12/11/2026 15:32	Roberto Nascimento	criou	espaco	2	
12/11/2026 15:32	Roberto Nascimento	criou	espaco	1	
12/11/2026 15:32	Roberto Nascimento	aprovou	usuario	8	
12/11/2026 15:32	Roberto Nascimento	aprovou	usuario	7	

O horário é o da carga dos dados; muda a cada execução do script 02.

5 VISÕES

Uma visão é uma consulta guardada no banco com nome, usada como se fosse uma tabela. Serve para não repetir consultas complexas e, como mostra a seção 9, para entregar a um usuário só a parte dos dados que ele pode ver.

5.1 vw_situacao_unidades

Cada apartamento com o responsável, as cobranças e o valor em atraso, o que está a vencer e a situação: em dia ou inadimplente.

```
SELECT * FROM vw_situacao_unidades ORDER BY apartamento;
```

unidade_id	apartamento	fracao_ideal	responsavel	cobrancas_em_atraso	valor_em_atraso	a_vencer	situacao
1	101	13.5000	Roberto Nascimento	1	320.00	0	INADIMPLENTE
2	102	13.5000	Bruno Cardoso	1	320.00	0	INADIMPLENTE

unidade_id	apartamento	fracao_ideal	responsavel	cobrancas_em_atraso	valor_em_atraso	a_vencer	situacao
3	204	15.5000	João Silva	2	470.00	0	INADIMPLENTE
4	301	13.5000	Ana Beatriz Rocha	1	320.00	0	INADIMPLENTE
5	302	13.5000	João Silva	2	640.00	0	INADIMPLENTE
6	410	15.5000	Marina Duarte	2	546.66	0	INADIMPLENTE
7	502	15.0000		0	0	0	EM DIA

5.2 vw_balancete_mensal

Receitas, despesas, saldo do mês e saldo acumulado, por condomínio e mês: a prestação de contas.

```
SELECT to_char(mes, 'MM/YYYY') AS mes, receitas, despesas, saldo_do_mes, saldo_acumulado
FROM vw_balancete_mensal ORDER BY vw_balancete_mensal.mes;
```

mes	receitas	despesas	saldo_do_mes	saldo_acumulado
07/2026	2146.67	1331.17	815.50	815.50
08/2026	1826.67	2311.17	-484.50	331.00
09/2026	0	1331.17	-1331.17	-1000.17

5.3 vw_moradores

Os moradores sem CPF, telefone nem senha, e com o e-mail mascarado: é o que a portaria enxerga.

```
SELECT * FROM vw_moradores ORDER BY apartamento;
```

id	nome	apartamento	tipo_ocupacao	reside	status	email
10	Roberto Nascimento	101	PROPRIETARIO	true	ATIVO	s***@smartcondo.com
7	Bruno Cardoso	102	INQUILINO	true	ATIVO	b***@smartcondo.com
5	João Silva	204	PROPRIETARIO	true	ATIVO	m***@smartcondo.com
6	Ana Beatriz Rocha	301	PROPRIETARIO	true	ATIVO	a***@smartcondo.com
11	João Silva	302	PROPRIETARIO	false	ATIVO	m***@smartcondo.com
8	Marina Duarte	410	PROPRIETARIO	true	ATIVO	m***@smartcondo.com
9	Pedro Henrique Lima	502	INQUILINO	true	AGUARDANDO_APROVACAO	p***@smartcondo.com

6 FUNÇÕES E PROCEDIMENTOS

A **função** devolve um valor e pode ser usada dentro de um SELECT; o **procedimento** é executado com CALL, pode alterar dados e devolver resultados por parâmetros INOUT. As regras são as mesmas do sistema: a multa e os juros seguem o Código Civil (art. 1.336, § 1º) e o rateio não perde centavo.

Objeto	Parâmetros	Devolve / faz
fn_total_pago	cobrança	Quanto já foi abatido da cobrança (pagamentos menos multa e juros)
fn_encargos	cobrança, dia (hoje, se omitido)	Dias de atraso, multa (2%), juros (1% ao mês, dia a dia) e total
fn_rateio	condomínio, total	A parte de cada apartamento pela fração ideal; recusa se faltar fração ou se elas não somarem 100%
sp_gerar_taxa_mensal	condomínio, competência, valor, por fração?, dia do vencimento	Lança a taxa do mês para todos os apartamentos, sem repetir quem já tem
sp_registrar_pagamento	cobrança, forma, quem pagou, valor (opcional)	Registra o pagamento com a multa e os juros do dia, travando a cobrança (FOR UPDATE)

6.1 fn_encargos: multa e juros de hoje

```
SELECT c.id, c.descricao, c.valor, c.vencimento, e.* FROM cobrancas c CROSS JOIN LATERAL fn_encargos(c.id) e WHERE c.id IN (11, 13, 19) ORDER BY c.id;
```

id	descricao	valor	vencimento	dias_atraso	multa	juros	total
11	Taxa de condomínio 08/2026	320.00	2026-08-10	94	6.40	10.03	16.43
13	Taxa de condomínio 09/2026	320.00	2026-09-10	63	6.40	6.72	13.12
19	Uso do espaço Salão de Festas em 25/09/2026	150.00	2026-09-25	48	3.00	2.40	5.40

Exemplo: R$ 320,00 com 94 dias de atraso → multa de 2% (R$ 6,40) e juros de 1% ao mês por 94 dias (R$ 10,03).

6.2 fn_rateio: divisão pela fração ideal

Dividir R$ 1.000,01 pelas frações dá valores com mais de duas casas. A função arredonda cada parte para baixo e entrega os centavos que sobram às partes com maior resto (método do maior resto), com a função de janela ROW_NUMBER(): a soma fecha exatamente.

```
SELECT * FROM fn_rateio(1, 1000.01);
```

unidade_id	apartamento	fracao	valor
1	101	13.5000	135.00
2	102	13.5000	135.00
3	204	15.5000	155.01
4	301	13.5000	135.00
5	302	13.5000	135.00
6	410	15.5000	155.00
7	502	15.0000	150.00

```
SELECT SUM(valor) AS soma FROM fn_rateio(1, 1000.01);
```

soma
1000.01

6.3 sp_gerar_taxa_mensal: a taxa do mês para todos

Orçamento de R$ 2.240,00 rateado pela fração ideal, para o mês que vem:

```
CALL sp_gerar_taxa_mensal(1, (CURRENT_DATE + INTERVAL '1 month')::date, 2240.00, true);
```

geradas
7

```
SELECT u.numero AS apartamento, c.descricao, c.valor, c.vencimento, c.status
  FROM cobrancas c
  JOIN unidades u ON u.id = c.unidade_id
 WHERE c.competencia = date_trunc('month', CURRENT_DATE + INTERVAL '1 month')::date
   AND c.origem = 'TAXA'
 ORDER BY u.numero;
```

apartamento	descricao	valor	vencimento	status
101	Taxa de condomínio 12/2026	302.40	2026-12-10	ABERTA
102	Taxa de condomínio 12/2026	302.40	2026-12-10	ABERTA
204	Taxa de condomínio 12/2026	347.20	2026-12-10	ABERTA
301	Taxa de condomínio 12/2026	302.40	2026-12-10	ABERTA
302	Taxa de condomínio 12/2026	302.40	2026-12-10	ABERTA
410	Taxa de condomínio 12/2026	347.20	2026-12-10	ABERTA
502	Taxa de condomínio 12/2026	336.00	2026-12-10	ABERTA

Executado de novo, o procedimento gera **0** cobranças: ninguém é cobrado duas vezes no mesmo mês.

6.4 sp_registrar_pagamento

O apartamento 101 paga a taxa de setembro, vencida. O procedimento calcula a multa e os juros do dia e registra o pagamento; quem marca a cobrança como paga é o gatilho da seção 7.

```
CALL sp_registrar_pagamento(13, 'PIX', 10);

SELECT cobranca_id, valor, encargos, forma FROM pagamentos WHERE cobranca_id = 13;
```

cobranca_id	valor	encargos	forma
13	333.12	13.12	PIX

```
SELECT id, descricao, valor, status FROM cobrancas WHERE id = 13;
```

id	descricao	valor	status
13	Taxa de condomínio 09/2026	320.00	PAGA

7 GATILHOS

Um gatilho (trigger) é uma função que o banco executa sozinho quando uma tabela muda. Assim, a regra vale para qualquer programa que grave no banco, e não só para o sistema.

Gatilho	Tabela	Quando	O que faz
trg_valida_pagamento	pagamentos	BEFORE INSERT	Barra pagamento de cobrança cancelada ou negociada e pagamento acima do que falta
trg_quita_cobranca	pagamentos	AFTER INSERT	Marca a cobrança como PAGA quando o que foi pago cobre o valor
trg_log_cobranca	cobrancas	AFTER UPDATE (só se mudar situação, valor ou vencimento)	Grava a mudança em log_cobrancas, com o usuário do banco e a hora
trg_atualizado_em	cobrancas, unidades, reservas	BEFORE UPDATE	Atualiza a data da última alteração; a mesma função serve às três tabelas

No pagamento da seção 6.4, os gatilhos trabalharam em cadeia: o **trg_valida_pagamento** conferiu o pagamento, o **trg_quita_cobranca** mudou a cobrança para PAGA e essa mudança disparou o **trg_log_cobranca**:

```
SELECT cobranca_id, status_anterior, status_novo, usuario_banco FROM log_cobrancas;
```

cobranca_id	status_anterior	status_novo	usuario_banco
13	VENCIDA	PAGA	smartcondo

A função do log é SECURITY DEFINER: grava na tabela de log com os direitos do dono do banco. Assim, nenhum usuário precisa (nem consegue) escrever no log diretamente — só o gatilho escreve.

8 INTEGRIDADE, ÍNDICE E TRANSAÇÕES

8.1 O que o banco recusa

O script 06 tenta cinco operações proibidas. Cada tentativa fica num bloco DO com tratamento de exceção (EXCEPTION), que funciona como um ponto de salvamento: o erro é capturado, a mensagem aparece na aba Messages do pgAdmin e o script continua.

Tentativa	Quem barrou	Mensagem do banco
Pagar a cobrança 20, que entrou num acordo	Gatilho	A cobrança 20 está negociada: não recebe pagamento.
Pagar de novo a cobrança 13, já quitada	Gatilho	O pagamento abate 50.00, mas da cobrança 13 só falta 0.00.
Pagamento de valor negativo	CHECK	violates check constraint "ck_pagamento_encargos"
Cobrança para a unidade 999, que não existe	Chave estrangeira	violates foreign key constraint "cobrancas_unidade_id_fkey"

Tentativa	Quem barrou	Mensagem do banco
Segunda taxa do mesmo mês para o mesmo apartamento	Índice único	violates unique constraint "uq_cobranca_competencia"

8.2 Índice parcial

As cobranças consultadas no dia a dia são as abertas ou vencidas; as pagas só crescem com o tempo. O índice **ix_cobrancas_em_aberto** guarda só essas, ordenadas pelo vencimento, e fica pequeno mesmo com anos de histórico:

```
CREATE INDEX ix_cobrancas_em_aberto
    ON cobrancas (vencimento)
 WHERE status IN ('ABERTA', 'VENCIDA');
```

Com poucas linhas, o PostgreSQL prefere ler a tabela inteira; desligando essa opção (enable_seqscan = off), o EXPLAIN mostra que o índice serve à consulta de atraso:

```
Index Scan using ix_cobrancas_em_aberto on cobrancas  (cost=0.14..8.15 rows=1 width=12)
  Index Cond: (vencimento < CURRENT_DATE)
```

8.3 Transações

O script 06 começa com **BEGIN** e termina com **ROLLBACK**: gera as taxas, registra o pagamento e muda situações, e no fim tudo é desfeito. Depois do ROLLBACK, a cobrança 13 volta a estar VENCIDA e o log volta a ficar vazio — é a atomicidade da transação. Dentro do procedimento de pagamento, o SELECT ... **FOR UPDATE** trava a linha da cobrança até o fim da transação: se dois pagamentos da mesma cobrança chegarem ao mesmo tempo, o segundo espera o primeiro terminar e então é barrado pelo gatilho, em vez de os dois passarem.

9 SEGURANÇA E CONTROLE DE ACESSO

O script 05 cria três papéis (roles) sem login (NOLOGIN), que representam funções no condomínio; um usuário real recebe o papel com GRANT. Nenhuma senha aparece nos scripts. Antes, o script tira de PUBLIC (todos os usuários) o acesso às tabelas e a execução das funções e procedimentos.

Papel	Pode
smartcondo_sindico	Ler, incluir e alterar tudo e executar as funções e procedimentos; não apaga nada (o histórico é preservado) e não escreve no histórico nem no log
smartcondo_portaria	Visitantes, encomendas e veículos; dos usuários, só as colunas id, nome, papel, status e unidade (permissão por coluna); a visão vw_moradores; nada do financeiro
smartcondo_conselho	Só as visões vw_situacao_unidades e vw_balancete_mensal: o conselho fiscal confere as contas sem ver dados pessoais

O fim do script 05 mostra, com has_table_privilege, o que cada papel pode em cada objeto:

papel	objeto	consulta	insere	altera	apaga
smartcondo_conselho	cobrancas	false	false	false	false
smartcondo_conselho	pagamentos	false	false	false	false
smartcondo_conselho	vw_balancete_mensal	true	false	false	false
smartcondo_portaria	cobrancas	false	false	false	false
smartcondo_portaria	visitantes	true	true	true	false
smartcondo_portaria	vw_moradores	true	false	false	false
smartcondo_sindico	cobrancas	true	true	true	false
smartcondo_sindico	log_cobrancas	true	false	false	false
smartcondo_sindico	pagamentos	true	true	true	false

Recorte das 21 linhas do resultado

No script 06, o banco assume o papel da portaria (SET ROLE) e tenta ler o CPF dos usuários e a tabela de pagamentos. As duas leituras são barradas — “permission denied for table usuarios” e “permission denied for table pagamentos” —, enquanto a visão vw_moradores responde normalmente, com o e-mail mascarado.

10 CONCLUSÃO

O banco do SmartCondo foi conferido e está consistente com o sistema que o usa: os scripts criam exatamente a estrutura do back-end, executam sem erro em banco vazio e podem ser executados de novo. Sobre ele, a 2ª parte implementou 14 consultas que respondem a perguntas reais do condomínio, 3 visões, 3 funções, 2 procedimentos, 6 gatilhos, um índice parcial e um controle de acesso por papéis com permissão por tabela, por coluna e por visão. As regras mais importantes — não pagar o que foi negociado, não pagar duas vezes, não cobrar a taxa duas vezes no mês, guardar o histórico das cobranças — passam a ser garantidas pelo próprio banco, para qualquer programa que grave nele.

APÊNDICE A – SCRIPT 04_PROGRAMACAO.SQL

```
DROP VIEW IF EXISTS vw_situacao_unidades, vw_balancete_mensal, vw_moradores;
DROP PROCEDURE IF EXISTS sp_gerar_taxa_mensal, sp_registrar_pagamento;
DROP FUNCTION IF EXISTS fn_valida_pagamento, fn_quita_cobranca, fn_log_cobranca, fn_atualizado_em
CASCADE;
DROP FUNCTION IF EXISTS fn_rateio, fn_encargos, fn_total_pago;
DROP TABLE IF EXISTS log_cobrancas;
DROP INDEX IF EXISTS ix_cobrancas_em_aberto;

CREATE TABLE log_cobrancas (
    id serial PRIMARY KEY,
    cobranca_id integer NOT NULL REFERENCES cobrancas (id),
    status_anterior varchar(20),
    status_novo varchar(20),
    valor_anterior numeric(10,2),
    valor_novo numeric(10,2),
    vencimento_anterior date,
    vencimento_novo date,
    usuario_banco varchar(63) NOT NULL DEFAULT current_user,
    alterado_em timestamp with time zone NOT NULL DEFAULT now()
);

CREATE INDEX ix_cobrancas_em_aberto
    ON cobrancas (vencimento)
 WHERE status IN ('ABERTA', 'VENCIDA');

CREATE FUNCTION fn_total_pago(p_cobranca integer)
RETURNS numeric
LANGUAGE sql STABLE
AS $$
    SELECT COALESCE(SUM(valor - encargos), 0)
      FROM pagamentos
     WHERE cobranca_id = p_cobranca;
$$;

CREATE FUNCTION fn_encargos(p_cobranca integer, p_dia date DEFAULT CURRENT_DATE)
RETURNS TABLE (dias_atraso integer, multa numeric, juros numeric, total numeric)
LANGUAGE plpgsql STABLE
AS $$
DECLARE
    v_status status_cobranca;
    v_restante numeric;
    v_vencimento date;
    v_multa_pct numeric;
    v_juros_pct numeric;
BEGIN
    SELECT c.status, c.valor - fn_total_pago(c.id), c.vencimento, co.multa_atraso,
co.juros_mensais
      INTO v_status, v_restante, v_vencimento, v_multa_pct, v_juros_pct
      FROM cobrancas c
      JOIN unidades u ON u.id = c.unidade_id
      JOIN condominios co ON co.id = u.condominio_id
     WHERE c.id = p_cobranca;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'A cobrança % não existe.', p_cobranca;
    END IF;
```

```
    dias_atraso := GREATEST(p_dia - v_vencimento, 0);
    IF dias_atraso = 0 OR v_restante <= 0 OR v_status NOT IN ('ABERTA', 'VENCIDA') THEN
        dias_atraso := 0;
        multa := 0.00;
        juros := 0.00;
    ELSE
        multa := ROUND(v_restante * v_multa_pct / 100, 2);
        juros := ROUND(v_restante * v_juros_pct / 100 * dias_atraso / 30, 2);
    END IF;
    total := multa + juros;
    RETURN NEXT;
END;
$$;

CREATE FUNCTION fn_rateio(p_condominio integer, p_total numeric)
RETURNS TABLE (unidade_id integer, apartamento varchar, fracao numeric, valor numeric)
LANGUAGE plpgsql STABLE
AS $$
DECLARE
    v_faltam integer;
    v_soma numeric;
BEGIN
    IF p_total IS NULL OR p_total <= 0 THEN
        RAISE EXCEPTION 'O total a ratear precisa ser maior que zero.';
    END IF;

    SELECT COUNT(*) FILTER (WHERE u.fracao_ideal IS NULL), COALESCE(SUM(u.fracao_ideal), 0)
      INTO v_faltam, v_soma
      FROM unidades u
     WHERE u.condominio_id = p_condominio
       AND NOT u.pendente;

    IF v_faltam > 0 THEN
        RAISE EXCEPTION 'Faltam as frações ideais de % unidade(s).', v_faltam;
    END IF;
    IF ABS(v_soma - 100) > 0.01 THEN
        RAISE EXCEPTION 'As frações ideais somam %%%; precisam somar 100%%.', v_soma;
    END IF;

    RETURN QUERY
    WITH base AS (
        SELECT u.id, u.bloco, u.numero, u.fracao_ideal,
               p_total * 100 * u.fracao_ideal / v_soma AS exato
          FROM unidades u
         WHERE u.condominio_id = p_condominio
           AND NOT u.pendente
    ),
    piso AS (
        SELECT b.*, floor(b.exato) AS centavos, b.exato - floor(b.exato) AS resto
          FROM base b
    ),
    sobra AS (
        SELECT round(p_total * 100) - SUM(p.centavos) AS n
          FROM piso p
    )
    SELECT p.id, p.numero, p.fracao_ideal,
           ((p.centavos + CASE WHEN ROW_NUMBER() OVER (ORDER BY p.resto DESC, p.id) <= s.n
```

```
                                THEN 1 ELSE 0 END) / 100)::numeric(10,2)
      FROM piso p
     CROSS JOIN sobra s
     ORDER BY p.bloco, p.numero;
END;
$$;

CREATE PROCEDURE sp_gerar_taxa_mensal(
    p_condominio integer,
    p_competencia date,
    p_valor numeric,
    p_por_fracao boolean DEFAULT false,
    p_dia_vencimento integer DEFAULT 10,
    INOUT geradas integer DEFAULT NULL
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_mes date := date_trunc('month', p_competencia)::date;
    v_vencimento date;
BEGIN
    IF p_valor IS NULL OR p_valor <= 0 THEN
        RAISE EXCEPTION 'O valor da taxa precisa ser maior que zero.';
    END IF;
    IF p_dia_vencimento NOT BETWEEN 1 AND 28 THEN
        RAISE EXCEPTION 'O dia do vencimento vai de 1 a 28.';
    END IF;
    v_vencimento := v_mes + (p_dia_vencimento - 1);

    INSERT INTO cobrancas (unidade_id, competencia, descricao, valor, vencimento, status, origem)
    SELECT u.id,
           v_mes,
           'Taxa de condomínio ' || to_char(v_mes, 'MM/YYYY'),
           CASE WHEN p_por_fracao THEN r.valor ELSE p_valor END,
           v_vencimento,
           CASE WHEN v_vencimento < CURRENT_DATE THEN 'VENCIDA' ELSE 'ABERTA'
END::status_cobranca,
           'TAXA'
      FROM unidades u
      LEFT JOIN fn_rateio(p_condominio, CASE WHEN p_por_fracao THEN p_valor END) r
             ON p_por_fracao AND r.unidade_id = u.id
     WHERE u.condominio_id = p_condominio
       AND NOT u.pendente
       AND NOT EXISTS (
             SELECT 1
               FROM cobrancas c
              WHERE c.unidade_id = u.id
                AND c.competencia = v_mes
                AND c.origem = 'TAXA'
                AND c.status <> 'CANCELADA'
          );

    GET DIAGNOSTICS geradas = ROW_COUNT;
    RAISE NOTICE '% cobrança(s) gerada(s) para %.', geradas, to_char(v_mes, 'MM/YYYY');
END;
$$;

CREATE PROCEDURE sp_registrar_pagamento(
```

```
    p_cobranca integer,
    p_forma forma_pagamento,
    p_pago_por integer,
    p_valor numeric DEFAULT NULL,
    INOUT pagamento_id integer DEFAULT NULL
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_restante numeric;
    v_encargos numeric;
    v_valor numeric;
BEGIN
    SELECT c.valor - fn_total_pago(c.id)
      INTO v_restante
      FROM cobrancas c
     WHERE c.id = p_cobranca
       FOR UPDATE;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'A cobrança % não existe.', p_cobranca;
    END IF;

    SELECT e.total INTO v_encargos FROM fn_encargos(p_cobranca) e;
    v_valor := COALESCE(p_valor, v_restante + v_encargos);
    IF v_valor <= 0 THEN
        RAISE EXCEPTION 'O valor do pagamento precisa ser maior que zero.';
    END IF;

    INSERT INTO pagamentos (cobranca_id, pago_por_id, valor, encargos, forma, pago_em)
    VALUES (p_cobranca, p_pago_por, v_valor, LEAST(v_valor, v_encargos), p_forma, now())
    RETURNING id INTO pagamento_id;

    RAISE NOTICE 'Pagamento % registrado: % (sendo % de multa e juros).',
                 pagamento_id, v_valor, LEAST(v_valor, v_encargos);
END;
$$;

CREATE FUNCTION fn_valida_pagamento()
RETURNS trigger
LANGUAGE plpgsql
AS $$
DECLARE
    v_status status_cobranca;
    v_valor numeric;
    v_falta numeric;
BEGIN
    SELECT status, valor INTO v_status, v_valor
      FROM cobrancas
     WHERE id = NEW.cobranca_id
       FOR UPDATE;

    IF v_status IN ('CANCELADA', 'NEGOCIADA') THEN
        RAISE EXCEPTION 'A cobrança % está %: não recebe pagamento.',
                        NEW.cobranca_id, lower(v_status::text);
    END IF;

    v_falta := v_valor - fn_total_pago(NEW.cobranca_id);
```

```
    IF NEW.valor - NEW.encargos > v_falta THEN
        RAISE EXCEPTION 'O pagamento abate %, mas da cobrança % só falta %.',
                        NEW.valor - NEW.encargos, NEW.cobranca_id, v_falta;
    END IF;
    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_valida_pagamento
BEFORE INSERT ON pagamentos
FOR EACH ROW EXECUTE FUNCTION fn_valida_pagamento();

CREATE FUNCTION fn_quita_cobranca()
RETURNS trigger
LANGUAGE plpgsql
AS $$
BEGIN
    UPDATE cobrancas
       SET status = 'PAGA'
     WHERE id = NEW.cobranca_id
       AND status IN ('ABERTA', 'VENCIDA')
       AND fn_total_pago(id) >= valor;
    RETURN NULL;
END;
$$;

CREATE TRIGGER trg_quita_cobranca
AFTER INSERT ON pagamentos
FOR EACH ROW EXECUTE FUNCTION fn_quita_cobranca();

CREATE FUNCTION fn_log_cobranca()
RETURNS trigger
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = public
AS $$
BEGIN
    INSERT INTO log_cobrancas (cobranca_id, status_anterior, status_novo,
                               valor_anterior, valor_novo,
                               vencimento_anterior, vencimento_novo)
    VALUES (NEW.id, OLD.status, NEW.status,
            OLD.valor, NEW.valor,
            OLD.vencimento, NEW.vencimento);
    RETURN NULL;
END;
$$;

CREATE TRIGGER trg_log_cobranca
AFTER UPDATE ON cobrancas
FOR EACH ROW
WHEN (OLD.status IS DISTINCT FROM NEW.status
   OR OLD.valor IS DISTINCT FROM NEW.valor
   OR OLD.vencimento IS DISTINCT FROM NEW.vencimento)
EXECUTE FUNCTION fn_log_cobranca();

CREATE FUNCTION fn_atualizado_em()
RETURNS trigger
LANGUAGE plpgsql
```

```
AS $$
BEGIN
    NEW.atualizado_em := now();
    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_atualizado_em BEFORE UPDATE ON cobrancas
FOR EACH ROW EXECUTE FUNCTION fn_atualizado_em();

CREATE TRIGGER trg_atualizado_em BEFORE UPDATE ON unidades
FOR EACH ROW EXECUTE FUNCTION fn_atualizado_em();

CREATE TRIGGER trg_atualizado_em BEFORE UPDATE ON reservas
FOR EACH ROW EXECUTE FUNCTION fn_atualizado_em();

CREATE VIEW vw_situacao_unidades AS
SELECT u.id AS unidade_id,
       u.numero AS apartamento,
       u.fracao_ideal,
       (SELECT m.nome
          FROM usuarios m
         WHERE m.unidade_id = u.id
           AND m.papel = 'MORADOR'
           AND m.status = 'ATIVO'
         ORDER BY m.reside DESC, m.tipo_ocupacao, m.id
         LIMIT 1) AS responsavel,
       COUNT(c.id) FILTER (WHERE c.status IN ('ABERTA', 'VENCIDA')
                           AND c.vencimento < CURRENT_DATE) AS cobrancas_em_atraso,
       COALESCE(SUM(c.valor - COALESCE(p.pago, 0))
                FILTER (WHERE c.status IN ('ABERTA', 'VENCIDA')
                          AND c.vencimento < CURRENT_DATE), 0) AS valor_em_atraso,
       COALESCE(SUM(c.valor - COALESCE(p.pago, 0))
                FILTER (WHERE c.status IN ('ABERTA', 'VENCIDA')
                          AND c.vencimento >= CURRENT_DATE), 0) AS a_vencer,
       CASE WHEN COUNT(c.id) FILTER (WHERE c.status IN ('ABERTA', 'VENCIDA')
                                     AND c.vencimento < CURRENT_DATE) > 0
            THEN 'INADIMPLENTE'
            ELSE 'EM DIA'
       END AS situacao
  FROM unidades u
  LEFT JOIN cobrancas c ON c.unidade_id = u.id
  LEFT JOIN (SELECT cobranca_id, SUM(valor - encargos) AS pago
               FROM pagamentos
              GROUP BY cobranca_id) p
         ON p.cobranca_id = c.id
 WHERE NOT u.pendente
 GROUP BY u.id, u.numero, u.fracao_ideal;

CREATE VIEW vw_balancete_mensal AS
WITH receitas AS (
       SELECT u.condominio_id,
              date_trunc('month', p.pago_em AT TIME ZONE 'America/Campo_Grande')::date AS mes,
              SUM(p.valor) AS valor
         FROM pagamentos p
         JOIN cobrancas c ON c.id = p.cobranca_id
         JOIN unidades u ON u.id = c.unidade_id
        GROUP BY 1, 2
```

```
    ),
    gastos AS (
      SELECT condominio_id,
             date_trunc('month', data)::date AS mes,
             SUM(valor) AS valor
        FROM despesas
       WHERE inativo_em IS NULL
       GROUP BY 1, 2
    )
SELECT COALESCE(r.condominio_id, g.condominio_id) AS condominio_id,
       COALESCE(r.mes, g.mes) AS mes,
       COALESCE(r.valor, 0) AS receitas,
       COALESCE(g.valor, 0) AS despesas,
       COALESCE(r.valor, 0) - COALESCE(g.valor, 0) AS saldo_do_mes,
       SUM(COALESCE(r.valor, 0) - COALESCE(g.valor, 0))
           OVER (PARTITION BY COALESCE(r.condominio_id, g.condominio_id)
                 ORDER BY COALESCE(r.mes, g.mes)) AS saldo_acumulado
  FROM receitas r
  FULL JOIN gastos g
         ON g.condominio_id = r.condominio_id
        AND g.mes = r.mes;

CREATE VIEW vw_moradores AS
SELECT m.id,
       m.nome,
       u.numero AS apartamento,
       m.tipo_ocupacao,
       m.reside,
       m.status,
       regexp_replace(m.email, '^(.).*(@.*)$', '\1***\2') AS email
  FROM usuarios m
  JOIN unidades u ON u.id = m.unidade_id
 WHERE m.papel = 'MORADOR';
```

APÊNDICE B – SCRIPT 05_SEGURANCA.SQL

```
DO $$
DECLARE
    v_papel text;
BEGIN
    FOREACH v_papel IN ARRAY ARRAY['smartcondo_sindico', 'smartcondo_portaria',
'smartcondo_conselho'] LOOP
        IF EXISTS (SELECT 1 FROM pg_roles WHERE rolname = v_papel) THEN
            EXECUTE format('DROP OWNED BY %I', v_papel);
        ELSE
            EXECUTE format('CREATE ROLE %I NOLOGIN', v_papel);
        END IF;
    END LOOP;
END;
$$;

REVOKE ALL ON ALL TABLES IN SCHEMA public FROM PUBLIC;
REVOKE EXECUTE ON FUNCTION fn_total_pago, fn_encargos, fn_rateio FROM PUBLIC;
REVOKE EXECUTE ON PROCEDURE sp_gerar_taxa_mensal, sp_registrar_pagamento FROM PUBLIC;

GRANT USAGE ON SCHEMA public TO smartcondo_sindico, smartcondo_portaria, smartcondo_conselho;

GRANT SELECT, INSERT, UPDATE ON ALL TABLES IN SCHEMA public TO smartcondo_sindico;
GRANT USAGE ON ALL SEQUENCES IN SCHEMA public TO smartcondo_sindico;
REVOKE INSERT, UPDATE ON registros_alteracao, log_cobrancas FROM smartcondo_sindico;
GRANT EXECUTE ON FUNCTION fn_total_pago, fn_encargos, fn_rateio TO smartcondo_sindico;
GRANT EXECUTE ON PROCEDURE sp_gerar_taxa_mensal, sp_registrar_pagamento TO smartcondo_sindico;

GRANT SELECT ON unidades, espacos_comuns, reservas, veiculos_unidade, autorizacoes_visita
   TO smartcondo_portaria;
GRANT SELECT (id, nome, papel, status, unidade_id) ON usuarios TO smartcondo_portaria;
GRANT SELECT ON vw_moradores TO smartcondo_portaria;
GRANT SELECT, INSERT, UPDATE ON visitantes, encomendas, movimentacoes_veiculo
   TO smartcondo_portaria;
GRANT USAGE ON SEQUENCE visitantes_id_seq, encomendas_id_seq, movimentacoes_veiculo_id_seq
   TO smartcondo_portaria;

GRANT SELECT ON vw_situacao_unidades, vw_balancete_mensal TO smartcondo_conselho;

SELECT r.papel,
       t.objeto,
       has_table_privilege(r.papel, t.objeto, 'SELECT') AS consulta,
       has_table_privilege(r.papel, t.objeto, 'INSERT') AS insere,
       has_table_privilege(r.papel, t.objeto, 'UPDATE') AS altera,
       has_table_privilege(r.papel, t.objeto, 'DELETE') AS apaga
  FROM (VALUES ('smartcondo_sindico'), ('smartcondo_portaria'), ('smartcondo_conselho')) r
(papel)
 CROSS JOIN (VALUES ('cobrancas'), ('pagamentos'), ('usuarios'), ('visitantes'),
                    ('log_cobrancas'), ('vw_moradores'), ('vw_balancete_mensal')) t (objeto)
 ORDER BY r.papel, t.objeto;
```

APÊNDICE C – SCRIPT 06_DEMONSTRACAO.SQL

```
BEGIN;

SELECT * FROM vw_situacao_unidades ORDER BY apartamento;

SELECT to_char(mes, 'MM/YYYY') AS mes, receitas, despesas, saldo_do_mes, saldo_acumulado
  FROM vw_balancete_mensal
 ORDER BY vw_balancete_mensal.mes;

SELECT * FROM fn_rateio(1, 2240.00);

SELECT c.id,
       u.numero AS apartamento,
       c.descricao,
       c.valor,
       e.dias_atraso,
       e.multa,
       e.juros,
       c.valor - fn_total_pago(c.id) + e.total AS total_hoje
  FROM cobrancas c
  JOIN unidades u ON u.id = c.unidade_id
 CROSS JOIN LATERAL fn_encargos(c.id) e
 WHERE c.status IN ('ABERTA', 'VENCIDA')
 ORDER BY u.numero, c.vencimento;

CALL sp_gerar_taxa_mensal(1, (CURRENT_DATE + INTERVAL '1 month')::date, 2240.00, true);

SELECT u.numero AS apartamento, c.descricao, c.valor, c.vencimento, c.status
  FROM cobrancas c
  JOIN unidades u ON u.id = c.unidade_id
 WHERE c.competencia = date_trunc('month', CURRENT_DATE + INTERVAL '1 month')::date
   AND c.origem = 'TAXA'
 ORDER BY u.numero;

CALL sp_gerar_taxa_mensal(1, (CURRENT_DATE + INTERVAL '1 month')::date, 2240.00, true);

CALL sp_registrar_pagamento(13, 'PIX', 10);

SELECT id, descricao, valor, status FROM cobrancas WHERE id = 13;

SELECT id, cobranca_id, valor, encargos, forma FROM pagamentos WHERE cobranca_id = 13;

SELECT cobranca_id, status_anterior, status_novo, usuario_banco FROM log_cobrancas;

DO $$
BEGIN
    INSERT INTO pagamentos (cobranca_id, valor, forma, pago_em)
    VALUES (20, 410.00, 'PIX', now());
    RAISE NOTICE 'ERRO: o pagamento da cobrança negociada passou.';
EXCEPTION WHEN raise_exception THEN
    RAISE NOTICE 'Gatilho barrou: %', SQLERRM;
END;
$$;

DO $$
DECLARE
    v_pagamento integer;
```

```
BEGIN
    CALL sp_registrar_pagamento(13, 'PIX', 10, 50.00, v_pagamento);
    RAISE NOTICE 'ERRO: a cobrança foi paga duas vezes.';
EXCEPTION WHEN raise_exception THEN
    RAISE NOTICE 'Gatilho barrou: %', SQLERRM;
END;
$$;

DO $$
BEGIN
    INSERT INTO pagamentos (cobranca_id, valor, forma, pago_em)
    VALUES (14, -5.00, 'PIX', now());
    RAISE NOTICE 'ERRO: o pagamento negativo passou.';
EXCEPTION WHEN check_violation THEN
    RAISE NOTICE 'Restrição CHECK barrou: %', SQLERRM;
END;
$$;

DO $$
BEGIN
    INSERT INTO cobrancas (unidade_id, competencia, descricao, valor, vencimento, status, origem)
    VALUES (999, date_trunc('month', CURRENT_DATE), 'Unidade que não existe', 100.00,
            CURRENT_DATE + 10, 'ABERTA', 'TAXA');
    RAISE NOTICE 'ERRO: a cobrança de unidade inexistente passou.';
EXCEPTION WHEN foreign_key_violation THEN
    RAISE NOTICE 'Chave estrangeira barrou: %', SQLERRM;
END;
$$;

DO $$
BEGIN
    INSERT INTO cobrancas (unidade_id, competencia, descricao, valor, vencimento, status, origem)
    VALUES (1, '2026-07-01', 'Taxa repetida', 320.00, '2026-07-10', 'ABERTA', 'TAXA');
    RAISE NOTICE 'ERRO: a taxa repetida passou.';
EXCEPTION WHEN unique_violation THEN
    RAISE NOTICE 'Índice único barrou: %', SQLERRM;
END;
$$;

SET LOCAL enable_seqscan = off;

EXPLAIN
SELECT id, unidade_id, vencimento
  FROM cobrancas
 WHERE status IN ('ABERTA', 'VENCIDA')
   AND vencimento < CURRENT_DATE;

SET LOCAL ROLE smartcondo_portaria;

SELECT nome, apartamento, email FROM vw_moradores ORDER BY apartamento;

DO $$
DECLARE
    v_cpf text;
BEGIN
    SELECT cpf INTO v_cpf FROM usuarios LIMIT 1;
    RAISE NOTICE 'ERRO: a portaria leu o CPF.';
EXCEPTION WHEN insufficient_privilege THEN
```

```
    RAISE NOTICE 'Permissão barrou: %', SQLERRM;
END;
$$;

DO $$
BEGIN
    PERFORM 1 FROM pagamentos LIMIT 1;
    RAISE NOTICE 'ERRO: a portaria leu os pagamentos.';
EXCEPTION WHEN insufficient_privilege THEN
    RAISE NOTICE 'Permissão barrou: %', SQLERRM;
END;
$$;

RESET ROLE;

ROLLBACK;

SELECT id, descricao, status FROM cobrancas WHERE id = 13;
```